

Big Data and Data Engineering

Sistema de reservas de aerolínea: panel de analítica gerencial

Segundo parcial — Anexo: bitácora de uso de asistentes de IA

Yslen Natalia Moreno Gutierrez
Oscar Mauricio Peñuela Guerrero

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 21 de octubre de 2026

Índice

1. Integrante A: backend, frontend y despliegue **3**

1.1. 20 de octubre de 2026: plan, base de datos, backend y páginas 3

1.2. 21 de octubre de 2026: verificación y documento 4

2. Integrante B: datos, EMR y analítica **5**

3. Qué verificamos por nuestra cuenta **6**

1. Integrante A: backend, frontend y despliegue

1.1. 20 de octubre de 2026: plan, base de datos, backend y páginas

Hora	Prompt (resumen o textual)	Respuesta del asistente	Qué hicimos con ella
07:20	"En este parcial somos 2 personas, y nos dimos cuenta que podemos hacer este plan y tener todo el parcial, dime qué tal" (adjuntamos el plan en PDF).	Revisión del plan. Señaló riesgos en la regla C5, en cómo sumar la capacidad, en los tipos del Parquet, en el tamaño del cluster de EMR y en la transacción de R2.	Los usamos como lista de chequeo para el desarrollo.
07:22	"¿Podemos hacer todo el proceso y luego ver cómo hacemos los cambios?"	Separó lo que conviene hacer antes de pagar EMR o al escribir el endpoint de reservas de lo que puede esperar.	Lo seguimos.
07:26	"Puedo darte acceso a AWS para hacer la parte del A."	Respondió que no podía conectarse a AWS y recomendó no compartir credenciales en el chat. Propuso guiarnos paso a paso.	Hicimos nosotros todos los pasos en la consola.
07:28	"Quiero que me vayas dando un paso a paso … hasta qué punto puedo llegar sin necesitar lo de él."	Plan por fases de la persona A y el único punto que dependía de B (el endpoint de analítica).	Organizamos el trabajo con esas fases.
07:45	"Antes de seguir, ¿todo esto no podía hacerse en AWS?"	Comparó desarrollar en local y directamente en AWS.	Escogimos trabajar directo en AWS.
07:51	Capturas de RDS, EC2 y EMR de la cuenta.	Encontró dos clusters de EMR de otra actividad y advirtió que el harness revisa toda la cuenta.	Lo tuvimos en cuenta para R10 y R11.
07:57–08:35	Capturas, una por una, del formulario de creación de la RDS.	En cada pantalla indicó qué cambiar: PostgreSQL en vez de Aurora, una sola zona, db.t4g.micro en vez de m7g.large, gp3 de 20 GB en vez de io2 de 400 GB, Database Insights estándar, sin monitoreo mejorado, sin acceso público y base inicial `airline`.	Creamos la RDS `airline-db` con esa configuración.
08:35	"Ya la creé."	Pasos para lanzar la EC2 t3.small con `LabInstanceProfile` y los puertos 22 y 8000.	Lanzamos la EC2 `airline-backend`.
08:43	"Necesito parar aquí y luego volver, ¿debo apagar algo?"	Detener (no terminar) la EC2 y la RDS, y advirtió que la RDS se enciende sola a los 7 días.	Apagamos todo.
14:16	"Volví, ¿qué debo hacer para continuar?"	Regla del grupo de seguridad (5432 solo desde la EC2) y prueba de conexión con `psql`.	Configuramos la regla.
14:42	Error `password authentication failed for user .ec2-user"`.	Explicó que el comando se partió en dos líneas al pegarlo.	Repetimos el comando en una línea y conectó.

Hora	Prompt (resumen o textual)	Respuesta del asistente	Qué hicimos con ella
14:44	Error `syntax error at or near "psql"` al crear las tablas.	Explicó que pegamos el comando `psql` estando ya dentro de la base.	Creamos las tablas `flights` y `reservations`.
14:45	"Ya me sale que se creó."	Código de `main.py` (FastAPI) con pool de conexiones, `UPDATE` atómico dentro de una transacción y respuestas 404 y 409. Antes de dárnoslo lo probó con el harness contra un PostgreSQL local.	Lo copiamos a la EC2 y probamos en `/docs`.
14:52–15:01	"Se me cerró la sesión", "no entiendo qué hacer en post reservations", errores 422 en `/docs`.	Cómo reactivar el entorno de Python y cómo llenar el cuerpo JSON en `/docs`.	Probamos: 201, 201 y 409 para un vuelo de 2 cupos.
15:01	"Sí, ya sirvió."	Archivo de servicio de `systemd`.	Lo instalamos.
15:04	El navegador mostraba `ERR_CONNECTION_REFUSED`.	Detectó que la terminal seguía esperando el final del bloque porque el `EOF` tenía espacios al inicio.	Repetimos el bloque sin sangría y el servicio quedó activo.
15:06	"Listo, ya sale."	Las páginas `index.html` y `manager.html` con los `data-testid` del anexo A.3, probadas con el harness (R8 y R9) en local.	Las copiamos a la EC2.
15:22–15:27	Errores 422 al crear un vuelo de prueba en `/docs`.	Explicó la validación de la ruta (`AAA-BBB`) y de la capacidad.	Probamos el flujo completo: buscar, reservar y ver el código de confirmación.
15:29	"Ahora sigamos con la parte del B hasta que quede completo."	Advirtió que debíamos coordinar con el integrante B para no duplicar corridas de EMR. Preparó un job de PySpark de referencia, lo validó en local contra el oráculo, escribió un endpoint de analítica y dio los pasos para generar y subir los datos.	No los usamos en el despliegue: la parte B ya la estaba haciendo el integrante B con su propio código.

1.2. 21 de octubre de 2026: verificación y documento

Hora	Prompt (resumen o textual)	Respuesta del asistente	Qué hicimos con ella
20:30	"Ya esta parte la hizo mi compañero, ¿cómo pruebo que todo funcione?"	Lista de pruebas en orden: EMR, archivos en S3, tablas en Glue y Athena, endpoint y harness completo.	La seguimos en CloudShell.

Hora	Prompt (resumen o textual)	Respuesta del asistente	Qué hicimos con ella
20:37–20:40	Salidas de `aws emr list-clusters`, `list-steps` y `list-instances`.	Confirmó los dos clusters del parcial (m4.large, step COMPLETED, 24 horas en total). Encontró que el cluster viejo usaba `r8g.xlarge`, lo que hace fallar R11 hasta el 23 de noviembre.	Lo documentamos en la sección 6.
20:50–20:52	Salidas de `aws s3 ls`, `aws glue get-tables` y un error al pegar SQL en CloudShell.	Una función para consultar Athena desde CloudShell.	Obtuvimos 1 997 430 filas y 24 meses.
20:53	Valores de ocupación de Athena.	Los comparó con el oráculo del harness: coinciden.	—
20:54	Respuesta del endpoint `/analytics/occupancy`.	Confirmó que es igual a Athena. Dio el `config.yaml` y el `cost_projection.yaml`.	Corrimos el harness: 58 de 70.
21:00	"¿Ya no hay más revisiones que hacer?"	Pendientes: Elastic IP, prueba de reinicio, R8 y R9 con navegador y contenido del repositorio.	Los anotamos como tareas antes de la entrega.
21:03	"Antes de seguir, quiero que me digas qué documentos se deben entregar."	Lista de entregables según la sección 7 del enunciado.	—
21:11–21:32	Pedimos el documento final en LaTeX, la portada y esta bitácora.	Borradores en LaTeX con los datos que debíamos confirmar marcados.	Los revisamos y completamos los datos pendientes.

2. Integrante B: datos, EMR y analítica

Fecha	Prompt (textual)	Respuesta del asistente	Qué hicimos con ella
19 oct, 22:33	"Me gustaría que dividieras el proyecto en fases, para realizar paso por paso y entender todo el proyecto."	División del trabajo en fases.	Seguimos el trabajo por fases.
20 oct, 22:33	"Bueno, ahora sí hagamos el trabajo de persona B, pero antes de empezar, ¿recuerdas el laboratorio de los taxis? Dime si la guía que me hiciste es parecida."	Comparación de la guía de la persona B con el laboratorio de taxis del curso.	Usamos lo que ya conocíamos del laboratorio para la parte B.
20 oct, 22:34	"O sea, mejor la dejo prendida."	Comparó el costo de dejar la RDS y la EC2 encendidas (alrededor de US$1 al día) con el trabajo de reconfigurar la IP cada vez que se reiniciaba el lab.	Decidimos dejarlas encendidas mientras trabajábamos. Después asignamos una IP elástica, con lo que ya se pueden apagar sin que cambie la dirección.

3. Qué verificamos por nuestra cuenta

- Cada configuración de AWS la revisamos en la consola después de crearla (clase de la RDS, Multi-AZ, almacenamiento, acceso público, reglas de los grupos de seguridad).
- El backend lo probamos a mano en `/docs` y en el navegador, y luego con el harness.
- Los números de la parte analítica los comparamos tres veces: en Athena, en el endpoint y contra el oráculo del harness.
- El puntaje y las fallas del documento salen de nuestra propia corrida del harness en CloudShell.